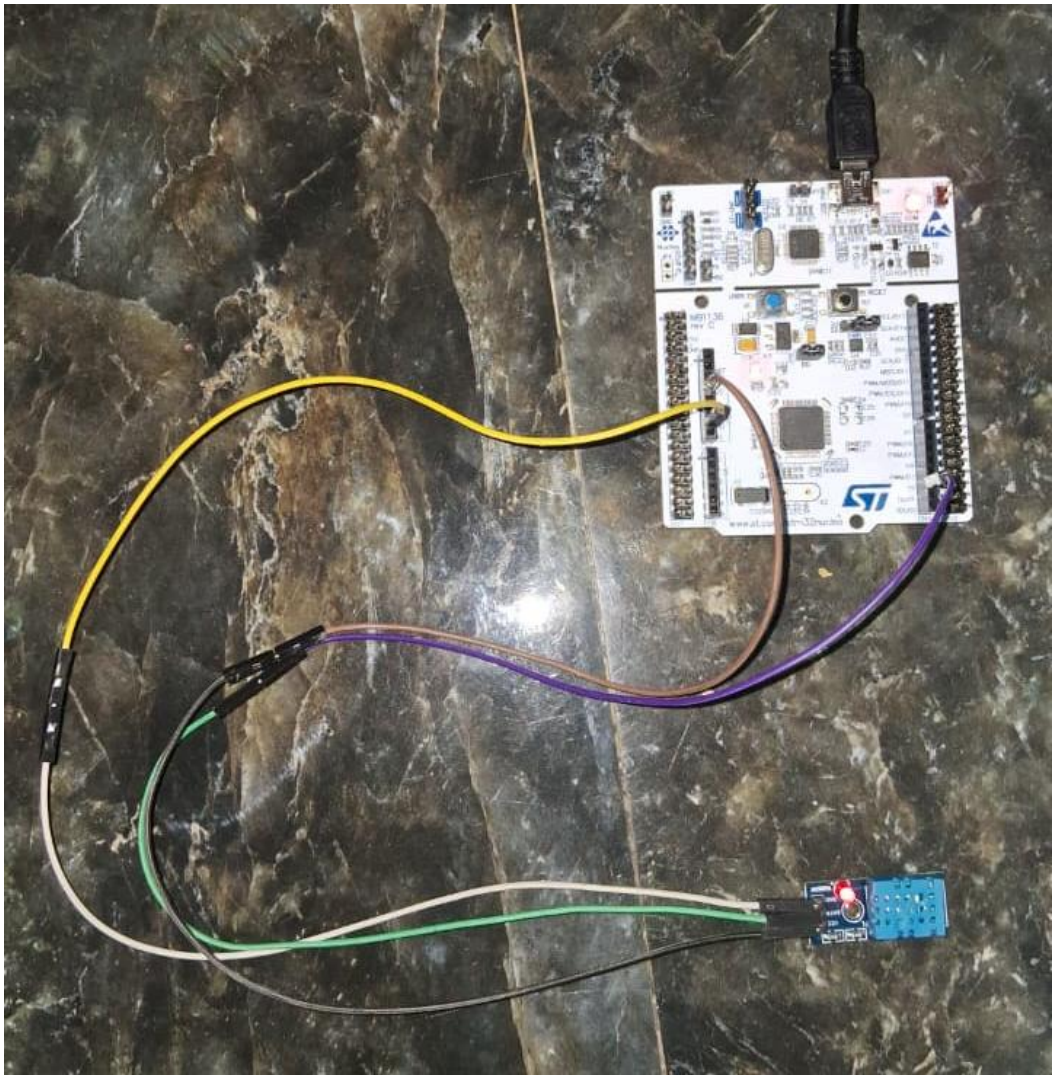
Design and Implementation of a Cloud-Connected Real-Time Closed-Loop Control System for Industrial Furnaces

WORK FLOW OUR PROJECT

HARDWARE SETUP

1.DHT11 TO STM32 NUCLEO-F103RB

The DHT11 temperature and humidity sensor is interfaced with the STM32 NUCLEO-F103RB using a single-wire data pin connected to a GPIO input. The STM32 reads real-time environmental data from the DHT11 and processes it for monitoring and control operations in the system.



CODEING PART

DHT11 TO STM32 NUCLEO-F103RB

CODE :

File Edit Sketch Tools Help

Nucleo-64

sketch_nov14a.ino

```
// DHT11 Settings
#define DHTPIN PA10     // D2 pin on STM32 (PA10)
#define DHTTYPE DHT11

DHT dht(DHTPIN, DHTTYPE);

void setup() {
  Serial.begin(9600);
  delay(1000);

  Serial.println("STM32 + DHT11 Temperature & Humidity Test");
  dht.begin();
}

void loop() {
  float humidity = dht.readHumidity();
  float temperature = dht.readTemperature();  // °C

  // Check if reading failed
  if (isnan(humidity) || isnan(temperature)) {
    Serial.println("Failed to read from DHT11 sensor!");
    delay(2000);
    return;
  }

  // Print output
  Serial.print("Temperature: ");
  Serial.print(temperature);
  Serial.print(" °C  |  Humidity: ");
  Serial.print(humidity);
  Serial.println(" %");

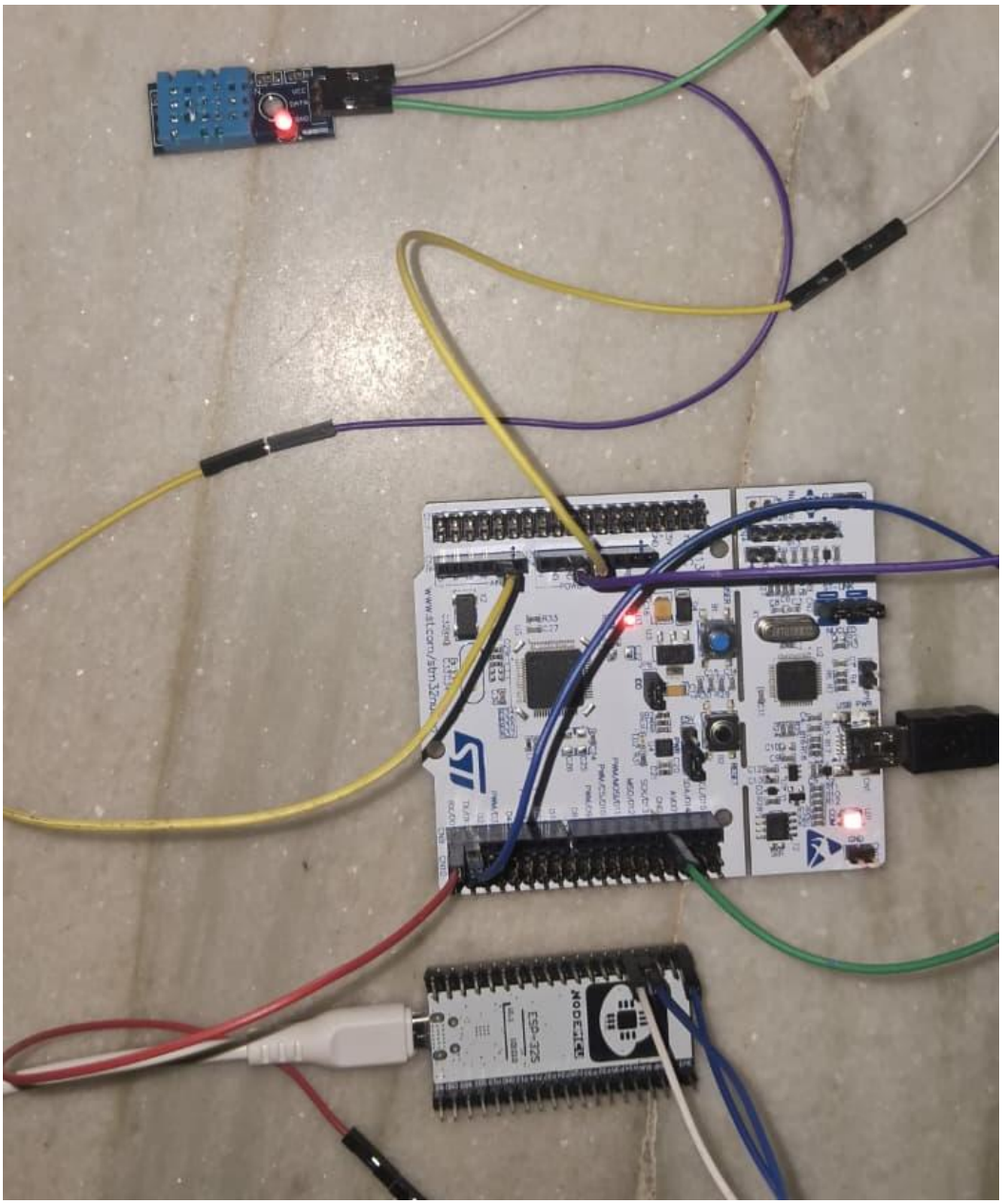
delay(2000);
}
```

OUTPUUT:

```
19:10:34.535 -> Temperature: 30.60 °C  |  Humidity: 65.50 %
19:10:36.594 -> Temperature: 30.80 °C  |  Humidity: 65.30 %
19:10:38.625 -> Temperature: 30.80 °C  |  Humidity: 65.20 %
19:10:40.653 -> Temperature: 30.80 °C  |  Humidity: 65.10 %
19:10:42.672 -> Temperature: 30.60 °C  |  Humidity: 65.10 %
19:10:44.704 -> Temperature: 30.80 °C  |  Humidity: 65.00 %
19:10:46.733 -> Temperature: 30.90 °C  |  Humidity: 64.90 %
```

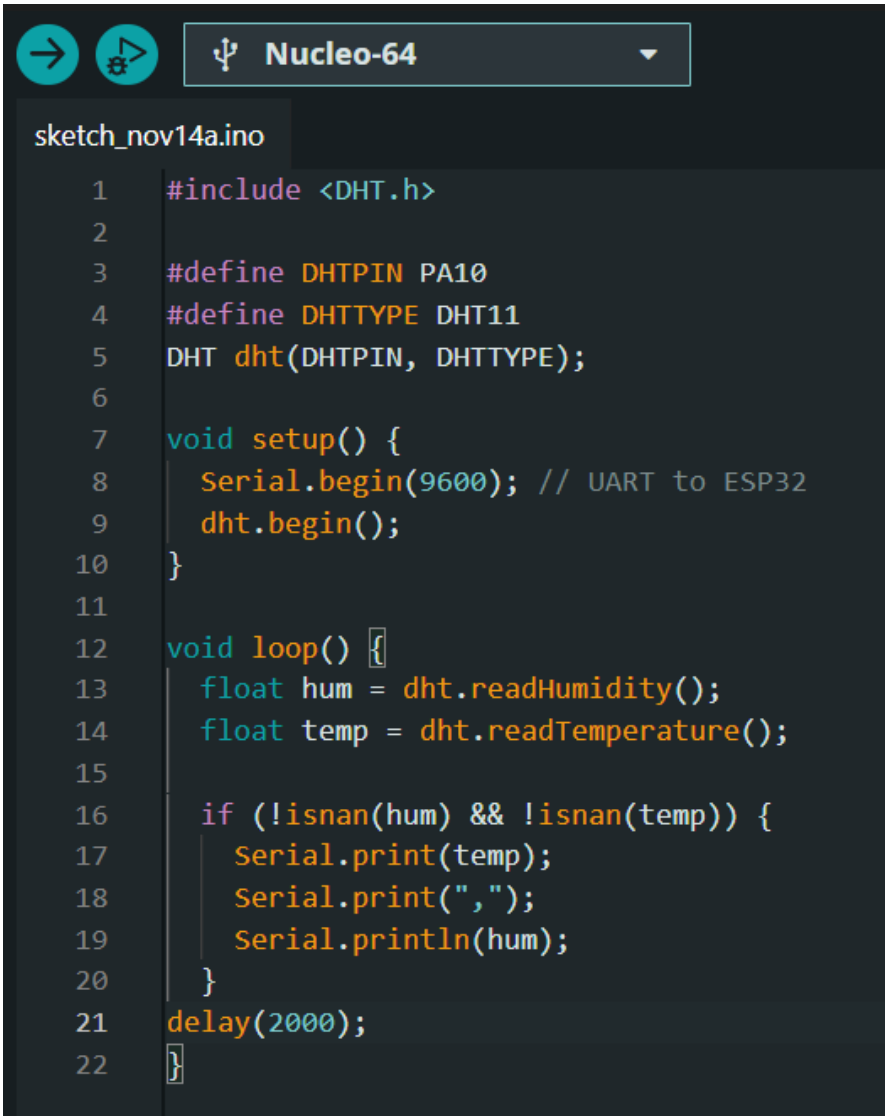
2.STM32 NUCLEO-F103RB TO NODEMCU 32S

The STM32 NUCLEO-F103RB is connected to the NodeMCU-32S using UART communication, where the STM32 performs real-time sensor processing and sends the data to the ESP32 module. The NodeMCU-32S uploads the received data to the cloud through Wi-Fi, enabling remote monitoring and control of the system.

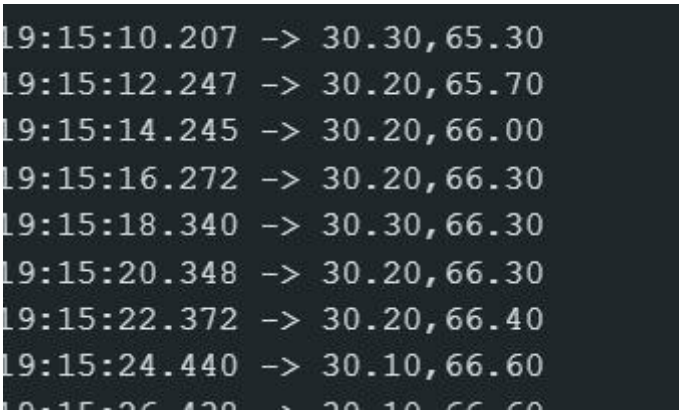


CODEING PART

CODE:



OUTPUT:



NODEMCU to cloud connect:

The NodeMCU connects to the cloud using built-in Wi-Fi, allowing sensor data received from the STM32 to be uploaded to online platforms such as Blynk or Firebase. This enables real-time remote monitoring and control of the system from any internet-connected device.

```
#define BLYNK_TEMPLATE_ID "TMPL3yt8nTbC-"
#define BLYNK_TEMPLATE_NAME "FURNACE TRACKER"
#define BLYNK_AUTH_TOKEN "YPN5vKv-H6aHPiXqLXpggxKWf_ahQ4sz"

// Uncomment this line for Serial debug output
#define BLYNK_PRINT Serial

#include <WiFi.h>
#include <WiFiClient.h>
#include <BlynkSimpleEsp32.h>
#include <DHT.h>

char auth[] = BLYNK_AUTH_TOKEN;

// Your WiFi credentials
char ssid[] = "naveen";
char pass[] = "naveenrs";

// DHT11 Setup
#define DHTPIN 4       // GPIO4 (D4 pin on NodeMCU-32S)
#define DHTTYPE DHT11
DHT dht(DHTPIN, DHTTYPE);

// Blynk Virtual Pins
#define VPIN_TEMP V0
#define VPIN_HUMI V1
#define VPIN_STATUS V2

// Fan control pin
#define FAN_PIN 5      // GPIO5 → controls relay/fan

// Thresholds
#define TEMP_THRESHOLD 35.0    // °C
#define HUM_THRESHOLD 70.0     // %

void setup()
{
  Serial.begin(115200);
  delay(1000);

  Serial.println("Connecting to WiFi...");
  Blynk.begin(auth, ssid, pass);
  dht.begin();

  // Set fan pin as OUTPUT and turn OFF initially
  pinMode(FAN_PIN, OUTPUT);
  digitalWrite(FAN_PIN, LOW);
}

void loop()
{
```

```ino
void loop()
{
  Blynk.run();
  readAndSendData();
  delay(3000); // Read every 3 seconds
}

void readAndSendData()
{
  float temp = dht.readTemperature();  // °C
  float hum = dht.readHumidity();

  // Check if DHT11 gives valid readings
  if (isnan(temp) || isnan(hum)) {
    Serial.println("Failed to read from DHT11 sensor!");
    return;
  }

  // Print to Serial Monitor
  Serial.print("Temperature: ");
  Serial.print(temp);
  Serial.print(" °C | Humidity: ");
  Serial.print(hum);
  Serial.print(" %");

  // Determine status and control fan
  String status;
  if (temp > TEMP_THRESHOLD || hum > HUM_THRESHOLD) {
    status = "Abnormal";
    digitalWrite(FAN_PIN, HIGH);  // Turn fan ON
  } else {
    status = "Normal";
    digitalWrite(FAN_PIN, LOW);   // Turn fan OFF
  }

  Serial.print(" | Status: ");
  Serial.println(status);

  // Send to Blynk Cloud
  Blynk.virtualWrite(VPIN_TEMP, temp);
  Blynk.virtualWrite(VPIN_HUMI, hum);
  Blynk.virtualWrite(VPIN_STATUS, status);
}
```

OUTPUT:

```
05:13:36.515 ->       ___  __           __
05:13:36.515 ->      / _ )/ /_ _____  / /__
05:13:36.515 ->     / _  / / // / _ \/  '_/
05:13:36.533 ->    /____/_/\_, /_//_/_/\_\
05:13:36.533 ->           /___/ v1.3.2 on ESP32
05:13:36.533 ->
05:13:36.533 ->    #StandWithUkraine    https://bit.ly/swua
05:13:36.533 ->
05:13:36.533 ->
05:13:36.533 -> [2645] Connecting to blynk.cloud:80
05:13:37.214 -> [3360] Ready (ping: 202ms).
```

THIS IS THE FINAL OUPUT FOR OUR PROJECT:

```
23:31:31.286 -> Temperature: 27.90 °C | Humidity: 80.10 %
23:31:31.358 -> Fan ON
23:31:33.327 -> Temperature: 27.90 °C | Humidity: 80.20 %
23:31:33.360 -> Fan ON
23:31:35.360 -> Temperature: 27.90 °C | Humidity: 80.20 %
23:31:35.392 -> Fan ON
23:31:37.376 -> Temperature: 27.90 °C | Humidity: 80.10 %
23:31:37.440 -> Fan ON
23:31:39.401 -> Temperature: 27.90 °C | Humidity: 80.10 %
23:31:39.472 -> Fan ON
```

CLOUD MONITORING SYSTEM

The cloud dashboard continuously receives real-time temperature and humidity data sent from the NodeMCU-32S and displays it through interactive widgets, charts, and status indicators. This cloud platform enables remote monitoring of furnace conditions from anywhere, providing live updates, historical trends, and alerts for abnormal temperature changes.

FURNACE TRACKER

Offline

Naveen My organization - 7162ZF

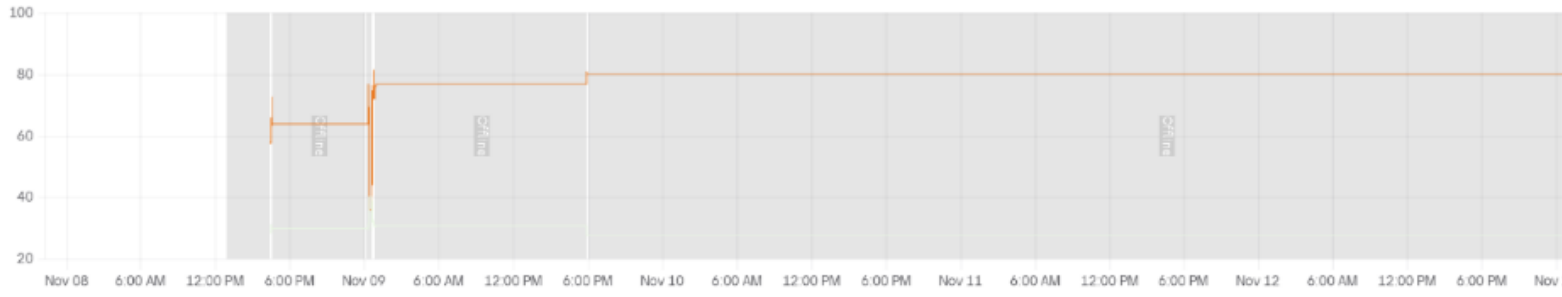
Edit

Live 1h 6h 1d 1w 1mo 3mo 6mo 1y

TEMPARATURE	HUMIDITY	STATUS
30.2 °C	53.5	Normal

Custom Chart

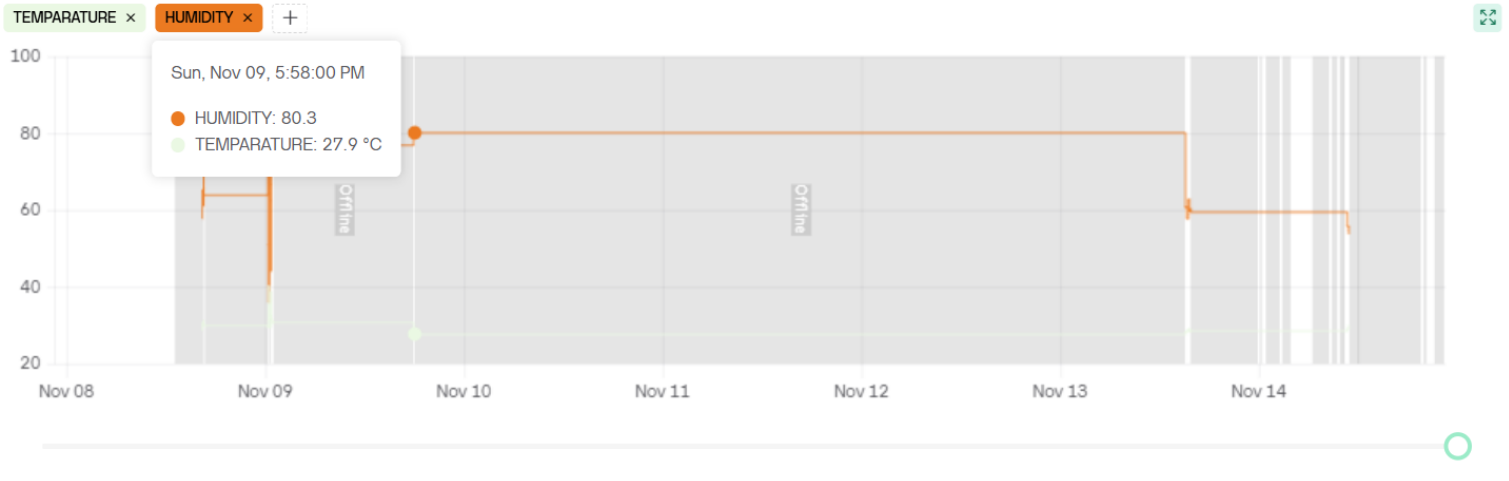
TEMPARATURE × HUMIDITY ×



Live 1h 6h 1d 1w 1mo 3mo 6mo 1y

Custom Chart

TEMPARATURE × HUMIDITY ×



THIS IS OVERALL OF OUR HARDWARE SETUP

